GLA UNIVERSITY

STUDY MATERIAL FOR STUDENTS

JavaScript Fundamentals

QnA - Version 1



Created by - Gautam Mukherjee

Assistant Professor

Email: gautam.mukherjee@gla.ac.in

Section A: Vanilla JavaScript Questions

Basic JavaScript Concepts

1 What are the different data types available in JavaScript? Explain them with examples.

ANSWER

JavaScript has **8 data types**. Seven are **primitive** (they store a single, simple value) and one is **non-primitive** (`object`, which can store collections of values).

Type	Meaning	Example
`string`	Text inside quotes	`"GLA"`, `'hello'`
`number`	Integers and decimals	`25`, `3.14`
`bigint`	Very large integers (ends with n)	`123456789012345678n`
`boolean`	Only true or false	`true`, `false`
`undefined`	Variable declared but no value given	`let x;`
`null`	Intentionally empty value	`let car = null;`
`symbol`	Unique identifier	`Symbol("id")`
`object`	Key-value data, arrays, functions, dates	`{name: "Riya"}`, `[1, 2, 3]`

```
let name = "Riya";            // string
let age = 20;                 // number
let big = 9007199254740993n;  // bigint
let isStudent = true;         // boolean
let city;                     // undefined
let car = null;               // null
let id = Symbol("id");        // symbol

let student = { name: "Riya", age: 20 };  // object
let marks = [78, 85, 92];                 // array (a type of object)

console.log(typeof name);   // "string"
console.log(typeof age);    // "number"
console.log(typeof null);   // "object"  <- famous old quirk of JavaScript
```

Key point: `typeof null` returns `"object"`. This is a historical bug in JavaScript, not a rule. `null` is a primitive.

2 What is the difference between var, let, and const in JavaScript?

ANSWER

All three are used to declare variables. `var` is the old way. `let` and `const` came in ES6 (2015) and are preferred today because they are safer.

Feature	var	let	const
Scope	Function scope	Block scope `{ }`	Block scope `{ }`
Re-declare in same scope	Allowed	Not allowed	Not allowed
Re-assign value	Allowed	Allowed	Not allowed
Must initialise at declaration	No	No	Yes
Hoisting	Hoisted, value is `undefined`	Hoisted, but in Temporal Dead Zone	Hoisted, but in Temporal Dead Zone

```
var a = 1;
var a = 2;          // OK with var
let b = 1;
// let b = 2;       // SyntaxError: already declared
const PI = 3.14;
// PI = 3.15;       // TypeError: assignment to constant variable

if (true) {
  var x = 10;       // function/global scoped
  let y = 20;       // only inside this block
}
console.log(x);     // 10
console.log(y);     // ReferenceError: y is not defined

const arr = [1, 2];
arr.push(3);        // OK: const stops re-assigning, not changing the contents
console.log(arr);   // [1, 2, 3]
```

Key point: Best practice: use `const` by default, use `let` when the value must change, and avoid `var`.

3 What is hoisting in JavaScript? Explain with a suitable example.

ANSWER

Hoisting means JavaScript moves **declarations** to the top of their scope before the code runs. Only the declaration is moved, **not the value assigned**.

- `var` variables are hoisted and given the value `undefined`.
- Function declarations are hoisted completely (name and body), so you can call them before writing them.
- `let` and `const` are hoisted too, but cannot be used before their line. The gap is called the **Temporal Dead Zone (TDZ)** and gives a `ReferenceError`.

```
console.log(a);      // undefined (declaration hoisted, value not yet assigned)
var a = 5;

sayHi();             // "Hi!"  (function declaration is fully hoisted)
function sayHi() {
  console.log("Hi!");
}

console.log(b);      // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

4 What is the DOM (Document Object Model), and how does JavaScript use it to interact with an HTML page?

ANSWER

The **DOM** is a tree-like representation of an HTML page that the browser creates. Every tag, text and attribute becomes a **node (object)** in this tree. JavaScript can read and change these nodes, so the page can update without reloading.

COMMON THINGS JAVASCRIPT DOES WITH THE DOM

- **Select** elements: `getElementById()`, `querySelector()`, `querySelectorAll()`
- **Change** content and style: `textContent`, `innerHTML`, `style`, `classList`
- **Create / remove** elements: `createElement()`, `appendChild()`, `remove()`
- **Respond to events**: `addEventListener("click", handler)`

```
<h1 id="title">Hello</h1>
<button id="btn">Click me</button>

<script>
  const title = document.getElementById("title");
  const btn = document.querySelector("#btn");

  btn.addEventListener("click", () => {
    title.textContent = "You clicked the button!";   // change text
    title.style.color = "green";                     // change style
  });

  const p = document.createElement("p");             // create a new element
  p.textContent = "I was added by JavaScript";
  document.body.appendChild(p);                      // add it to the page
</script>
```

5 What is the difference between a named function and an anonymous function in JavaScript?

ANSWER

A **named function** has a name after the `function` keyword. An **anonymous function** has no name and is usually stored in a variable or passed as a callback.

```
// Named function
function greet(name) {
  return "Hello " + name;
}
console.log(greet("Riya"));          // Hello Riya

// Anonymous function (stored in a variable)
const add = function (a, b) {
  return a + b;
};
console.log(add(2, 3));              // 5

// Anonymous function used as a callback
setTimeout(function () {
  console.log("Done after 1 second");
}, 1000);
```

- Named functions are easier to **debug** because the name shows in error messages.
- Named functions can call themselves (**recursion**) easily by using their name.

- Anonymous functions are short and handy for **callbacks** and one-time use.

6 What is a function expression in JavaScript? How is it different from a function declaration?

ANSWER

A **function expression** creates a function as part of an expression and usually stores it in a variable. A **function declaration** uses the `function` keyword with a name as a standalone statement.

Point	Function Declaration	Function Expression
Syntax	`function add(a, b) { }`	`const add = function(a, b) { };`
Hoisting	Fully hoisted, can be called before it is defined	Not usable before the line (variable rules apply)
Name	Name is required	Name is optional

```
sayHello();                       // "Hello!"  works because of hoisting
function sayHello() {             // function declaration
  console.log("Hello!");
}

// sayBye();                      // ReferenceError: Cannot access 'sayBye' before initialization
const sayBye = function () {      // function expression
  console.log("Bye!");
};
sayBye();                         // "Bye!"
```

7 What is an arrow function in JavaScript? Explain its syntax with an example.

ANSWER

An **arrow function** (ES6) is a shorter way to write a function using the `=>` symbol. It is mostly used for small functions and callbacks.

```
// Normal function
const square1 = function (n) { return n * n; };

// Arrow function
const square2 = (n) => n * n;        // one-line body: value is returned automatically
const greet = name => `Hello, ${name}`;   // single parameter: brackets are optional
const hello = () => "Hi!";           // no parameter: empty brackets
const sum = (a, b) => {              // multiple lines: use { } and return
  const total = a + b;
  return total;
};
const getUser = () => ({ name: "Riya" });  // returning an object: wrap it in ( )

console.log(square2(5));             // 25
console.log(greet("Aman"));          // Hello, Aman
console.log([1, 2, 3].map(n => n * 2));  // [2, 4, 6]
```

Key point: Arrow functions do **not** have their own `this`, they take `this` from the surrounding code. They also cannot be used as constructors (with `new`).

8 What is the difference between the == and === operators in JavaScript?

ANSWER

`==` is the **loose equality** operator. It converts (coerces) the types first and then compares values.
`===` is the **strict equality** operator. It compares both value **and** type with no conversion.

```
5 == "5"            // true   (string "5" is converted to number)
5 === "5"           // false  (number vs string)
0 == false          // true
0 === false         // false
null == undefined   // true
null === undefined  // false
NaN === NaN         // false  (NaN is never equal to anything, not even itself)
```

Key point: Always prefer `===` and `!==`. They avoid surprising bugs caused by automatic type conversion.

9 Can we store arrays, objects, and functions inside another array in JavaScript? Explain with an example.

ANSWER

Yes. A JavaScript array can hold values of any type, including numbers, strings, other arrays, objects and even functions, all in the same array.

```
const mixed = [
  10,                                  // number
  "hello",                             // string
  [1, 2, 3],                           // array inside array
  { name: "Riya", age: 20 },           // object
  function () { return "I am a function"; }   // function
];

console.log(mixed[2][1]);   // 2
console.log(mixed[3].name); // Riya
console.log(mixed[4]());    // I am a function
console.log(mixed.length);  // 5
```

10 Explain the purpose of the push() and pop() methods in JavaScript arrays, with examples.

ANSWER

Both methods work at the **end** of an array and change the original array.

- `push(item)` **adds** one or more items at the end and returns the new length.
- `pop()` **removes** the last item and returns that removed item.

```
const fruits = ["apple", "banana"];

const newLength = fruits.push("mango");
console.log(fruits);       // ["apple", "banana", "mango"]
console.log(newLength);    // 3

const removed = fruits.pop();
console.log(removed);      // "mango"
console.log(fruits);       // ["apple", "banana"]
```

> **Key point:** For the **start** of an array, use `unshift()` (add) and `shift()` (remove).

JavaScript Execution and Control Flow

11 What is the event loop in JavaScript, and how does it help JavaScript handle asynchronous operations?

ANSWER

JavaScript is **single-threaded**: it runs one task at a time on the **Call Stack**. Slow tasks such as timers, network calls and file reads are handed to the browser (Web APIs) or Node.js, so JavaScript does not get blocked. The **event loop** is the mechanism that keeps checking: "*Is the call stack empty? If yes, push the next waiting task onto it.*"

Part	Role
Call Stack	Runs the current code, one function at a time
Web APIs	Handle timers, fetch, DOM events in the background
Microtask Queue	Holds finished Promise callbacks (`.then`, `await`). Higher priority
Callback (Task) Queue	Holds finished `setTimeout`, click and other event callbacks
Event Loop	Moves tasks from the queues to the stack when the stack is empty

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

OUTPUT

```
Start
End
Promise
Timeout
```

Why this order? Normal code (Start, End) runs first. When the stack is empty, the event loop runs the microtask queue (Promise) first, and only then the callback queue (Timeout), even though the timer was 0 ms.

12 What is the Global Execution Context in JavaScript? Explain its role during program execution.

ANSWER

An **execution context** is the environment in which JavaScript code runs. The **Global Execution Context (GEC)** is the first and default context, created when the script starts. It sits at the bottom of the call stack until the program ends.

IT IS CREATED IN TWO PHASES

- **Creation (memory) phase:** creates the global object (`window` in browsers, `globalThis` in Node.js), sets `this`, and stores variables and functions in memory. `var` gets `undefined`, functions store their full code.
- **Execution phase:** runs the code line by line, assigns real values, and creates a new **function execution context** every time a function is called.

```
console.log(x);          // undefined  (memory phase already stored x)
var x = 10;

function show() {        // a new function execution context is created on call
  console.log("x is", x);
}
show();                  // x is 10
```

13 What is a for...of loop in JavaScript? How is it different from a traditional for loop?

ANSWER

A **for...of** loop (ES6) goes through the **values** of any iterable such as arrays, strings, Maps and Sets. It is shorter and easier to read than the traditional `for` loop, which needs an index counter.

```
const colors = ["red", "green", "blue"];

// Traditional for loop: you manage the index yourself
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}

// for...of loop: you directly get each value
for (const color of colors) {
  console.log(color);
}

for (const ch of "JS") console.log(ch);            // J, S
for (const [i, c] of colors.entries()) {           // when you also need the index
  console.log(i, c);                               // 0 red, 1 green, 2 blue
}
```

Traditional for	for...of
Needs counter, condition and increment	No counter needed
Gives you the index `i`	Gives you the value directly
More chance of off-by-one errors	Cleaner and safer
Works with anything you can index	Works with iterables only (not plain objects)

14 What is the difference between shallow copy and deep copy in JavaScript? Explain with a suitable example.

ANSWER

A **shallow copy** copies only the top level. Nested objects or arrays are still **shared** (same reference) with the original. A **deep copy** copies everything, including nested levels, so the copy is fully independent.

- Shallow copy methods: spread `{...obj}`, `Object.assign()`, `arr.slice()`, `[...arr]`
- Deep copy methods: `structuredClone(obj)` (modern and recommended), or `JSON.parse(JSON.stringify(obj))`

```
const original = { name: "Riya", address: { city: "Mathura" } };

// Shallow copy
```

```
const shallow = { ...original };
shallow.name = "Aman";            // top level: independent
shallow.address.city = "Delhi";   // nested: shared with original!
console.log(original.name);          // "Riya"   (safe)
console.log(original.address.city);  // "Delhi"  (changed by mistake)

// Deep copy
const deep = structuredClone(original);
deep.address.city = "Noida";
console.log(original.address.city);  // "Delhi"  (not affected)
console.log(deep.address.city);      // "Noida"
```

Key point: `JSON.parse(JSON.stringify())` loses functions, `undefined` and converts Dates to strings. `structuredClone()` keeps Dates but cannot copy functions.

15 What are the spread operator (...) and rest operator (...) in JavaScript? Explain their differences with examples.

ANSWER

Both use the same three dots `...` but do **opposite jobs**. **Spread** *expands* an array or object into individual items. **Rest** *collects* many items into one array or object.

Spread	Rest
Used where values are **given** (function call, array/object literal)	Used where values are **received** (function parameters, destructuring)
Expands one thing into many	Gathers many things into one
Example: `[...a, ...b]`	Example: `function f(...nums)`

```
// SPREAD: expand
const a = [1, 2], b = [3, 4];
const merged = [...a, ...b];                  // [1, 2, 3, 4]
const user = { name: "Riya", age: 20 };
const updated = { ...user, age: 21 };         // { name: "Riya", age: 21 }
console.log(Math.max(...[5, 9, 2]));          // 9

// REST: collect
function sum(...nums) {                       // nums is an array of all arguments
  return nums.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4));                 // 10

const [first, ...others] = [10, 20, 30, 40]; // first = 10, others = [20, 30, 40]
const { name, ...remaining } = user;          // name = "Riya", remaining = { age: 20 }
```

16 What is an object in JavaScript? Explain how to create an object and access its properties, with an example.

ANSWER

An **object** is a collection of **key-value pairs** (called properties). It is used to represent real-world things such as a student, a car or a product. If a property holds a function, it is called a **method**.

WAYS TO CREATE AN OBJECT

- **Object literal** (most common): `const obj = { }`

- `new Object()` constructor
- **Constructor function** or **class** with `new` (useful to create many similar objects)

```
// 1) Object literal
const student = {
  name: "Aman",
  age: 19,
  greet() {                              // method
    return `Hi, I am ${this.name}`;
  }
};

// 2) new Object()
const s2 = new Object();
s2.name = "Neha";

// 3) class
class Car {
  constructor(brand) { this.brand = brand; }
}
const c1 = new Car("Tata");

// Accessing properties
console.log(student.name);        // dot notation: "Aman"
console.log(student["age"]);      // bracket notation: 19
console.log(student.greet());     // "Hi, I am Aman"

student.course = "BTech";         // add a property
student.age = 20;                 // update a property
delete student.age;               // remove a property
```

17 What is a for...in loop in JavaScript? How is it commonly used to iterate over an object's properties?

ANSWER

A **for...in** loop goes through the **keys (property names)** of an object. Inside the loop, use `object[key]` to get each value.

```
const student = { name: "Aman", age: 19, course: "BCA" };

for (const key in student) {
  console.log(key + ": " + student[key]);
}
```

OUTPUT

```
name: Aman
age: 19
course: BCA
```

Be careful: `for...in` also visits **inherited** properties from the prototype. Use `Object.hasOwn()` to filter only the object's own properties.

```
const parent = { family: "Sharma" };
const child = Object.create(parent);   // child inherits from parent
child.name = "Aman";

for (const key in child) {
  if (Object.hasOwn(child, key)) console.log(key);   // prints only: name
}
```

Key point: Do not use `for...in` on arrays (order is not guaranteed and it gives indexes as strings). Use `for...of` for arrays. For objects, `Object.keys()`, `Object.values()` or `Object.entries()` with `for...of` is often cleaner.

18 What is the Object constructor or built-in Object type in JavaScript? Explain its purpose with an example.

ANSWER

`Object` is JavaScript's **built-in base type**. Almost everything in JavaScript (arrays, functions, dates) inherits from it. The `Object()` constructor can create objects, and the `Object` itself provides many **helpful static methods** to work with objects.

Method	Purpose
`Object.keys(obj)`	Returns array of keys
`Object.values(obj)`	Returns array of values
`Object.entries(obj)`	Returns array of [key, value] pairs
`Object.assign(target, src)`	Copies properties into a target object
`Object.freeze(obj)`	Makes object read-only
`Object.create(proto)`	Creates object with a chosen prototype
`Object.fromEntries(pairs)`	Builds an object from [key, value] pairs

```
const car = new Object();           // same as const car = {};
car.brand = "Tata";
car.model = "Nexon";

console.log(Object.keys(car));      // ["brand", "model"]
console.log(Object.values(car));    // ["Tata", "Nexon"]
console.log(Object.entries(car));   // [["brand","Tata"], ["model","Nexon"]]

const full = Object.assign({}, car, { year: 2024 });
console.log(full);                  // { brand: "Tata", model: "Nexon", year: 2024 }

Object.freeze(car);
car.brand = "Maruti";               // ignored (silently, or error in strict mode)
console.log(car.brand);             // "Tata"
```

Scope, Functions, and Advanced Concepts

19 What is lexical scope in JavaScript? Explain how it determines variable accessibility.

ANSWER

Lexical scope means a variable's accessibility is decided by **where it is written in the code**, not by where the function is called. An inner function can use variables of its outer functions, but an outer function cannot use variables of its inner functions. JavaScript looks for a variable in the current scope first, then moves outward step by step. This chain is called the **scope chain**.

```
const school = "GLA";                   // global scope
```

```
function outer() {
  const dept = "CSE";                    // outer function scope

  function inner() {
    const name = "Riya";                 // inner function scope
    console.log(name, dept, school);     // Riya CSE GLA  (looks outward: inner -> outer -> global)
  }

  inner();
  // console.log(name);                  // ReferenceError: inner variables are not visible here
}

outer();
// console.log(dept);                    // ReferenceError: dept lives only inside outer()
```

20 What is a closure in JavaScript? Explain its working with a practical, real-life example.

ANSWER

A **closure** is created when an inner function **remembers** the variables of its outer function, even after the outer function has finished running. It is the practical result of lexical scope.

Real-life example: a bank account. The balance must be private. Nobody should change it directly, only through deposit and withdraw. A closure gives us this privacy.

```
function bankAccount(initialBalance) {
  let balance = initialBalance;          // private variable (not accessible outside)

  return {
    deposit(amount) {
      balance += amount;
      return balance;
    },
    withdraw(amount) {
      if (amount > balance) return "Insufficient funds";
      balance -= amount;
      return balance;
    },
    getBalance() {
      return balance;
    }
  };
}

const account = bankAccount(1000);
console.log(account.deposit(500));     // 1500
console.log(account.withdraw(200));    // 1300
console.log(account.getBalance());     // 1300
console.log(account.balance);          // undefined  (cannot touch it directly)
```

Simple counter example: each call to `createCounter()` gets its own separate `count`.

```
function createCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const c1 = createCounter();
console.log(c1(), c1(), c1());    // 1 2 3
```

```
const c2 = createCounter();
console.log(c2());               // 1  (separate closure, separate count)
```

Key point: Uses of closures: data privacy, counters, event handlers, `setTimeout` callbacks, memoization and function factories.

21 What is a prototype in JavaScript, and how does it support inheritance?

ANSWER

Every JavaScript object has a hidden link to another object called its **prototype**. If a property or method is not found on the object itself, JavaScript looks for it in the prototype, then in the prototype's prototype, and so on. This chain is the **prototype chain** and it ends at `null`. This is how objects **inherit** properties and methods from other objects.

```
function Student(name) {
  this.name = name;
}
// Put the shared method on the prototype (stored only once in memory)
Student.prototype.introduce = function () {
  return "I am " + this.name;
};

const s1 = new Student("Riya");
const s2 = new Student("Aman");
console.log(s1.introduce());                      // I am Riya
console.log(s2.introduce());                      // I am Aman
console.log(s1.hasOwnProperty("introduce"));      // false (found via the prototype)
console.log(Object.getPrototypeOf(s1) === Student.prototype);   // true

// Inheritance through the prototype chain
function GradStudent(name, thesis) {
  Student.call(this, name);                       // reuse parent constructor
  this.thesis = thesis;
}
GradStudent.prototype = Object.create(Student.prototype);
GradStudent.prototype.constructor = GradStudent;

const g = new GradStudent("Neha", "AI in Web Apps");
console.log(g.introduce());                       // I am Neha  (inherited from Student)
```

The modern `class` and `extends` keywords are just a **cleaner syntax** over this same prototype system.

```
class Student {
  constructor(name) { this.name = name; }
  introduce() { return "I am " + this.name; }
}
class GradStudent extends Student {
  constructor(name, thesis) { super(name); this.thesis = thesis; }
}
console.log(new GradStudent("Neha", "AI").introduce());   // I am Neha
```

22 How does object-oriented programming (OOP) in JavaScript differ from OOP in languages such as Java and C++?

ANSWER

JavaScript supports OOP too, but its approach is different. Java and C++ are **class-based**: a class is a blueprint and objects are created from it. JavaScript is **prototype-based**: objects inherit directly from other objects. The `class` keyword in JavaScript (ES6) is only **syntactic sugar** over prototypes.

Feature	JavaScript	Java / C++
Inheritance model	Prototype-based (objects inherit from objects)	Class-based (classes inherit from classes)
Typing	Dynamic, types are checked at run time	Static, types are checked at compile time
Objects	Can be created without a class, and can be changed at run time	Must be created from a class with a fixed structure
Access modifiers	No `private` keyword. Uses `#field` for private and conventions like `_name`	`public`, `private`, `protected` keywords
Multiple inheritance	Not allowed with classes (mixins can be used)	C++ allows it. Java uses interfaces
Method overloading	Not supported. A later function with the same name replaces the earlier one	Supported
Interfaces / abstract classes	Not built in (TypeScript adds them)	Built in
Memory management	Automatic garbage collection	Java: automatic. C++: manual

```
class Account {
  #balance = 0;                     // private field (note the #)

  constructor(owner) { this.owner = owner; }
  deposit(amount) { this.#balance += amount; }
  get balance() { return this.#balance; }
}

const acc = new Account("Riya");
acc.deposit(500);
console.log(acc.balance);           // 500
// console.log(acc.#balance);       // SyntaxError: private field

// Objects can be changed at run time (not possible in Java/C++)
acc.bonus = true;
console.log(acc.bonus);             // true
```

Section B: Backend Setup - Node.js and NPM

Node.js Fundamentals

1 What is Node.js, and why is it used for backend development?

ANSWER

Node.js is an open-source **runtime environment** that lets you run JavaScript **outside the browser**, for example on a server. It is built on Chrome's **V8 JavaScript engine**, so it is fast.

WHY IT IS POPULAR FOR BACKEND DEVELOPMENT

- **Same language everywhere:** JavaScript for both frontend and backend (for example, the MERN stack).
- **Non-blocking, event-driven I/O:** can handle thousands of requests at the same time without waiting for slow tasks.
- **Fast:** V8 compiles JavaScript to machine code.
- **Huge ecosystem:** more than a million packages available through NPM.
- **Good for:** REST APIs, real-time apps (chat, live notifications), streaming and microservices.

```
// server.js  -> run with:  node server.js
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Node.js!");
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

2 What is NPM (Node Package Manager)? What are some alternatives to NPM?

ANSWER

NPM is the default **package manager** of Node.js. It comes installed with Node.js. It helps you install, update and remove ready-made libraries (called **packages**) from the NPM online registry, and it also runs scripts for your project.

```
npm install express        # install a package
npm uninstall express      # remove a package
npm update                 # update packages
npm run start              # run a script defined in package.json
npm -v                     # check version
```

ALTERNATIVES TO NPM

Tool	Highlights
Yarn	Made by Facebook. Fast, supports workspaces and offline cache
pnpm	Saves disk space by sharing packages between projects. Very fast

Tool	Highlights
Bun	All-in-one runtime and package manager, very fast installs

3 Which command is used to initialize an empty Node.js project? Explain the purpose of the command.

ANSWER

The command is `npm init`. It creates a **package.json** file in your project folder. This file stores details about your project and its packages.

```
mkdir my-app
cd my-app
npm init          # asks questions one by one (name, version, description, entry point, ...)
npm init -y       # skips the questions and uses default values
```

Purpose: `npm init` prepares the folder as a proper Node.js project so that you can install packages, define scripts and share the project with others.

4 What is the purpose of the package.json file in a Node.js project?

ANSWER

package.json is the **identity card and instruction sheet** of a Node.js project. It is a JSON file that tells Node and NPM what the project is, which packages it needs and how to run it.

- Stores project details: name, version, description, author, license.
- Lists all **dependencies** with their versions, so anyone can rebuild the project using `npm install`.
- Defines **scripts** (shortcuts for commands such as `npm start`).
- Defines the **entry point** (`main`) of the application.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Demo Node.js project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "author": "Student Name",
  "license": "ISC",
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "~3.1.0"
  }
}
```

5 What are dependencies in a Node.js project? How are they defined and managed in package.json?

ANSWER

Dependencies are the external packages (libraries) that your project needs in order to work. NPM downloads them into the **node_modules** folder and records them in **package.json**.

Type	Meaning	Install command
`dependencies`	Needed when the app **runs** (for example express, mongoose)	`npm install express`
`devDependencies`	Needed only during **development** (for example nodemon, jest)	`npm install nodemon --save-dev`
`peerDependencies`	Package expects the **host project** to provide it (used by library authors)	Added manually

```
npm install express            # adds to "dependencies"
npm install -D nodemon         # adds to "devDependencies"
npm install                    # installs everything already listed in package.json
npm uninstall express          # removes it from node_modules and package.json
npm update express             # updates within the allowed version range
npm list                       # shows installed packages
```

- **node_modules/** holds the downloaded code. Never upload it to GitHub (add it to `.gitignore`).
- **package-lock.json** records the exact versions installed, so every developer gets the same result.

6 What is the difference between the caret (^) and tilde (~) symbols when specifying package versions in package.json? Explain with examples.

ANSWER

Versions follow **MAJOR.MINOR.PATCH** (for example `4.18.2`). The symbols decide which updates NPM is allowed to install.

- **MAJOR**: big changes that may break old code
- **MINOR**: new features that still work with old code
- **PATCH**: small bug fixes

Written as	Name	Allowed updates	Example range for 4.18.2
`^4.18.2`	Caret	Minor and patch updates (not major)	`>=4.18.2` and `<5.0.0`
`~4.18.2`	Tilde	Patch updates only	`>=4.18.2` and `<4.19.0`
`4.18.2`	Exact	No updates	Only `4.18.2`

```
"dependencies": {
  "express": "^4.18.2",    // can become 4.19.0 or 4.21.1, but never 5.0.0
  "mongoose": "~7.5.0",    // can become 7.5.3, but never 7.6.0
  "lodash": "4.17.21"      // always exactly 4.17.21
}
```

Key point: For versions below 1.0.0, the caret is stricter: `^0.2.3` allows only `>=0.2.3 <0.3.0`, because in 0.x versions even a MINOR change is treated as a possibly breaking change.